

# Computation on NumPy Arrays

## Chapters 6-12

### The slowness of loops

The default implementation of Python (known as CPython) performs some operations **very slowly**.

This is partly due to the interpreted nature of the language that allows flexible types and renders efficient compilation to machine code impossible.

Python can be especially slow when small operations are being repeated on large inputs. For instance, looping over arrays to operate on each element.

For example, suppose we have an array of values and we would like to compute the *reciprocal* ( $1/x$ ) for each value  $x$ . A straightforward implementation would be:

In [40]:

```
import numpy as np
rng = np.random.default_rng(seed=1701)

def compute_reciprocals(values):
    output = np.empty(len(values))
```

```
for i in range(len(values)):
    output[i] = 1.0/values[i]
return output
```

```
values = rng.integers(1, 10, size=5)
compute_reciprocals(values)
```

Out[40]:

```
array([0.11111111, 0.25      , 1.        , 0.33333333, 0.125      ])
```

The above implementation may feel very natural to someone from a C or Java background. However, it is surprisingly slow in Python. Let us measure the execution time with `%timeit` magic on a very large input:

In [41]:

```
big_array = rng.integers(1, 100, size=10000000)
%timeit compute_reciprocals(big_array)
```

```
7.78 s ± 50.9 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

The reason the above operation is slow is mainly due to the type checking Python must do after each calculation. Indeed, after each loop cycle, Python examines the object's type and performs a lookup of the function to run for that type. If the code was compiled instead of interpreted, this specification would have been known before runtime and would run much faster.

## Introducing Ufuncs

NumPy provides a convenient interface to statically typed arrays. Known as **vectorized** operations.

For simple operations, like element-wise division, vectorization is as simple as using Python arithmetic. Indeed, NumPy *overloads* many of Python's operators to perform vectorized computations on arrays.

To compute reciprocals on a NumPy array, we would just use `1.0/values`. This would perform an element-wise division on the entire array and produce another array with the results:

In [42]:

```
print(compute_reciprocals(values))  
print(1.0/values)
```

```
[0.11111111 0.25      1.          0.33333333 0.125      ]  
[0.11111111 0.25      1.          0.33333333 0.125      ]
```

Let us time the runtime of reciprocals using vectorized division:

In [43]:

```
%timeit (1.0/big_array)
```

```
9.4 ms ± 142 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

## How much faster is the above implementation when compared to the loop-based one?

Vectorized operations are implemented via *ufuncs*.

Before we saw an operation between a scalar and an array, but we can also operate between two arrays:

In [44]:

```
np.arange(5) / np.arange(1, 6)
```

Out[44]:

```
array([0.          , 0.5          , 0.66666667, 0.75          , 0.8          ])
```

Ufunc operations also work on multidimensional arrays:

In [45]:

```
x = np.arange(9).reshape((3, 3))  
print(2**x)
```

```
[[ 1  2  4]  
 [ 8 16 32]  
 [64 128 256]]
```

**Exercise:** Create a program that initializes a 4x4 matrix `a` containing all ones, a 4x4 identity matrix `i`, and produces a matrix containing the element-wise subtraction of `i` from `a`. You must use vectorized operations to do this.

In [46]:

```
# Answer:
a = np.ones((4,4))
i = np.eye(4)
print(a - i)
```

```
[[0.  1.  1.  1.]
 [1.  0.  1.  1.]
 [1.  1.  0.  1.]
 [1.  1.  1.  0.]]
```

## Array arithmetic

NumPy's ufuncs feel very natural to use because they overload Python's native operators:

In [47]:

```
x = np.arange(4)
print("x      =", x)
print("x + 5  =", x + 5)
print("x - 5  =", x - 5)
print("x * 2   =", x * 2)
print("x / 2   =", x / 2)
print("x // 2  =", x // 2) # floor division
print("-x     =", -x)
```

```
print("x ** 2 = ", x ** 2)
print("x % 2 = ", x % 2)
```

```
x          = [0 1 2 3]
x + 5       = [5 6 7 8]
x - 5       = [-5 -4 -3 -2]
x * 2       = [0 2 4 6]
x / 2       = [0.  0.5 1.  1.5]
x // 2      = [0 0 1 1]
-x          = [ 0 -1 -2 -3]
x ** 2      = [0 1 4 9]
x % 2       = [0 1 0 1]
```

You can also write compound expression using array arithmetic:

In [48]:

```
-(0.5*x + 1) ** 2
```

Out[48]:

```
array([-1.  , -2.25, -4.  , -6.25])
```

**Remember:** Arithmetic operators are just interfaces, convenient ways, to use functions. You can also invoke NumPy ufuncs using their real name:

In [ ]:

```
# Repeating the operations from above but using ufunc calls instead of arithmetic operators

print("x      =", x)
print("x + 5  =", np.add(x, 5))
print("x - 5  =", np.subtract(x, 5))
```

```
print("x * 2 =", np.multiply(x, 2))
print("x / 2 =", np.divide(x, 2))
print("x // 2 =", np.floor_divide(x, 2)) # floor division
print("-x      =", np.negative(x))
print("x ** 2 =", np.power(x, 2))
print("x % 2  =", np.mod(x, 2))
```

## Absolute value

Python's built-in function `abs` compute the absolute value of a number. The corresponding NumPy ufunc is `np.absolute` or the shorthand `np.abs`:

In [ ]:

```
x = np.array([-2, -1, 0, 1, 2])
print(np.absolute(x))
```

## Trigonometric ufuncs

NumPy implements a number of trigonometric functions:

In [ ]:

```
theta = np.linspace(0, np.pi, 3)
```

In [ ]:

```
# Angles sine, cosine, tangent

print("theta      =", theta)
print("sin(theta) =", np.sin(theta))
```

```
print("cos(theta) = ", np.cos(theta))
print("tan(theta) = ", np.tan(theta))
```

In [ ]:

```
# Inverse trigonometric functions

x = [-1, 0, 1]
print("x          = ", x)
print("arcsin(x) = ", np.arcsin(x))
print("arccos(x) = ", np.arccos(x))
print("arctan(x) = ", np.arctan(x))
```

## Element-wise vs matrix multiplication

The multiplication operator `*` when applied to NumPy arrays performs *element-wise* multiplication. This means the result is the product of each element pair of the input arrays.

On the other hand, to perform *matrix multiplication*, you can use `@`:

In [49]:

```
x = np.array([[1,2], [3,4]])
y = np.array([[2,2], [3,3]])

print(x*y)
```

```
[[ 2  4]
 [ 9 12]]
```

In [50]:

```
print(x@y)
```

```
[[ 8  8]  
 [18 18]]
```

## Chapter 7. Aggregations: min, max and everything in between

When exploring datasets, it is often useful to compute various summary statistics. The most common ones might be the *mean* and *standard deviation*. NumPy implements fast aggregation functions to work on arrays.

### Array sum

Python has a built-in `sum` function, but NumPy's `np.sum` is much faster:

In [51]:

```
big_array = rng.random(1000000)  
  
%timeit sum(big_array)  
%timeit np.sum(big_array)
```

```
36 ms ± 86.9 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)  
171 µs ± 771 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

### Minimum and maximum values

Python's built-in `min` and `max` functions correspond to NumPy's `np.min` and `np.max`. The latter implementations are much faster.

In [ ]:

```
np.min(big_array), np.max(big_array)
```

## Multidimensional aggregates

Aggregating multidimensional arrays can be applied in three ways: to all elements in the array, along rows, or along columns.

If unspecified, aggregations will **apply across all elements of a multidimensional array**:

In [52]:

```
M = rng.integers(0, 10, (3, 4))  
print(M)  
print(M.sum())
```

```
[[7 3 2 7]  
 [5 4 2 7]  
 [3 7 4 9]]  
60
```

## Axis

Aggregation functions take an additional argument, called `axis`. This argument specifies the dimension along which the aggregation is computed. For example, to find the minimum value within each *column*, we specify `axis=0`:

In [53]:

```
print(M.min(axis=0)) # Finds the minimum of each column
```

```
[3 3 2 7]
```

Similarly, we can find the maximum value of each row:

In [54]:

```
print(M.max(axis=1))
```

```
[7 7 9]
```

**Note:** The way `axis` is specified can be confusing. The keyword specifies the dimension of the array that will be *collapsed* rather than the dimension that will be returned. Specifying `axis=0` means that **each row** will be collapsed (aggregated) into a single number.

## Chapter 8. Broadcasting

*Broadcasting* is a set of rules by which NumPy facilitates vectorized operations.

For **arrays of the same size**, binary operations are performed on an **element-by-element** basis:

In [55]:

```
a = np.array([0, 1, 2])
b = np.array([5, 5, 5])
a + b
```

Out[55]:

```
array([5, 6, 7])
```

Broadcasting also works with a vector and a scalar:

In [56]:

```
a + 5
```

Out[56]:

```
array([5, 6, 7])
```

We can think of the above as an operation that *stretches* scalar 5 into an array with [5, 5, 5].

This idea can be extended to higher dimensions. Observe the result when adding a one-dimensional array to a two-dimensional array:

In [57]:

```
M = np.ones((3, 3))  
M
```

Out[57]:

```
array([[1., 1., 1.],  
       [1., 1., 1.],  
       [1., 1., 1.]])
```

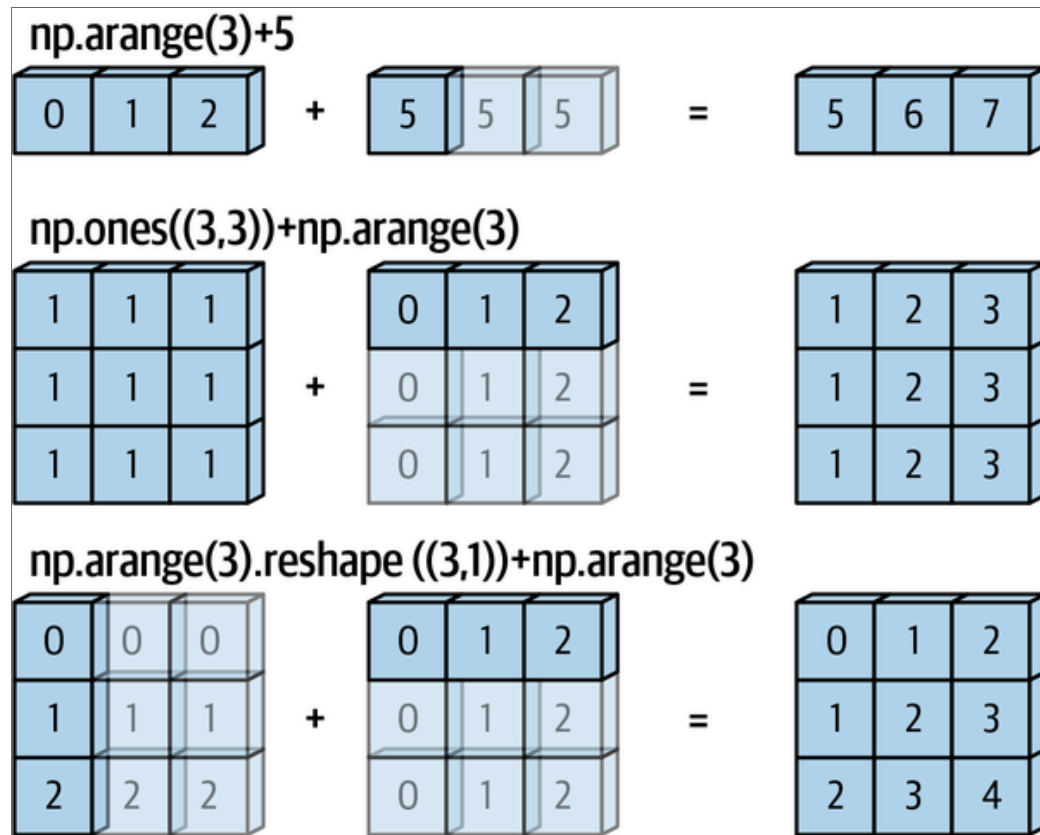
In [58]:

```
M + a
```

Out[58]:

```
array([[1., 2., 3.],  
       [1., 2., 3.],  
       [1., 2., 3.]])
```

Here is a visualization of broadcasting in NumPy:



## Chapter 9. Comparisons, masks and boolean logic

In this chapter, we will cover comparisons as vectorized operations, vector masks and boolean logic.

Comparison operators and Ufuncs

Similarly to arithmetic operators, comparison operators (such as `<` and `>`) are implemented in NumPy as Ufuncs. The result of a vectorized comparison operation is an array of *Booleans*:

In [59]:

```
x = np.array([1,2,3,4,5])
```

In [60]:

```
x < 3
```

Out[60]:

```
array([ True,  True, False, False, False])
```

In [61]:

```
x > 3
```

Out[61]:

```
array([False, False, False,  True,  True])
```

In [62]:

```
x <= 3
```

Out[62]:

```
array([ True,  True,  True, False, False])
```

In [63]:

```
x != 3
```

Out[63]:

```
array([ True,  True, False,  True,  True])
```

In [64]:

```
x == 3
```

Out[64]:

```
array([False, False,  True, False, False])
```

Like with arithmetic operators, comparisons can be applied to multidimensional arrays:

In [65]:

```
x = rng.integers(10, size=(3, 4))  
x
```

Out[65]:

```
array([[5, 5, 3, 3],  
       [8, 2, 3, 8],  
       [7, 6, 7, 3]])
```

In [66]:

```
x < 6
```

Out[66]:

```
array([[ True,  True,  True,  True],
       [False,  True,  True, False],
       [False, False, False,  True]])
```

## Counting results

Boolean values are treated as 0 if `False` or 1 if `True`. We can use this to *count* the number of specific observations in an array by *summing* the resulting Boolean values.

For instance, we can count how many elements of `x` are less than 5:

In [67]:

```
b_mask = x < 5
print(b_mask)

print(f"The number of elements less than 5 is {b_mask.sum()}")
```

```
[[False False  True  True]
 [False  True  True False]
 [False False False  True]]
The number of elements less than 5 is 5
```

## Boolean arrays as masks

We can use Boolean arrays as masks to retrieve a subset of a NumPy array. Let us take a look at array `x` from above:

In [68]:

```
print(x)
```

```
[[5 5 3 3]
 [8 2 3 8]
 [7 6 7 3]]
```

Suppose we want to filter all the values in `x` that are less than 5. This can be easily done with:

In [69]:

```
x < 5
```

Out[69]:

```
array([[False, False,  True,  True],
       [False,  True,  True, False],
       [False, False, False,  True]])
```

To effectively select these values from the array, we use the Boolean array from above to index `x`. This procedure is known as *masking*:

In [70]:

```
x[x < 5]
```

Out[70]:

```
array([3, 3, 2, 3, 3])
```